End-to-end Encryption (E2EE) Secure Instant Messenger

User Manual

Team19

1. System Requirements

- Operating System: Windows 11.0
- Python: Python 3.8 or higher must be installed.
- TLS Certificates: Put *cert.pem* and *key.pem* on the same folder as *server.py*
- Database Initialization: The database file (*secure_im.db*) and all relational tables are automatically generated by SQLAlchemy the very first time server.py is executed.
- Install required dependencies, open **Terminal / Command Prompt** and run:

```
pip install fastapi uvicorn sqlalchemy passlib[bcrypt] cryptography websocket-client requests
```

2. Installation & Starting the System

To properly test the 1:1 messaging locally, you must simulate two separate users. This requires setting up one server environment and two distinct client environments.

2.1. Recommended Folder Setup

Create three separate folders on your machine and distribute the files as follows:

- **Folder 1: Server_App**
  - server.py
  - cert.pem *(Must be in the exact same folder as server.py)*
  - key.pem *(Must be in the exact same folder as server.py)*
- **Folder 2: Client_1**
  - client.py

- **Folder 3: Client_2**
    - client.py

(Note: The clients must be run from separate folders so their local *client_state.json* encryption vaults do not overwrite each other).

2.2 Starting the Environment in VS Code

Step 1: Start the Server

- Open the Server_App folder in VS Code.
- Open **Terminal** and run:

  ```
  python server.py
  ```

- The server will start on https://127.0.0.1:8000 . You should see the message: `Secure E2EE IM Server starting...`
- Keep this terminal open. The server handles authentication, stores ciphertexts, and forwards messages.

Step 2: Start Client 1 (User A)

- Open a new VS Code window (File > New Window).
- Open the Client_1 folder in this new window.
- Open **Terminal** and run:
  `python client.py`
- A menu appears.

Step 3: Start Client 2 (User B)

- Open a **third** VS Code window.
- Open the Client_2 folder in this window.
- Open **terminal** and run:

  ```
  python client.py
  ```

- A menu appears.

3. Main Menu Overview

After launching the client, you will have to enter a passpharse that will be used to encrypt and decrypt local storage data.

- The passphrase does not have any restriction
- Can be seen as local storage password
- Have to input the same passphrase for each client launch

```
Enter local storage passphrase:
```

After that, you will be presented with the main menu:

```
=== Secure E2EE Instant Messenger ===
1. Login / Register
11. Exit
```

After logging in, you will see:

```
=== Secure E2EE Instant Messenger ===
Logged in as: a
2. Send friend request
3. View & respond to friend requests
4. Remove friend
5. List contacts & conversations
6. Verify Contact Fingerprint
7. Chat with contact
8. Block user
9. Unblock user
10. Logout
11. Exit
Choice:
```

Where {*a*} is the example of username or email that you used to login.

4. Application Features & Usage Guide

4.1 Login / Register (option 1)

1. Answer when asked "New user? (y/n)"

- If you are a new user:
    1. Enter a unique **username** or **email**
    2. Enter a strong **password** (minimum 12 characters, max 72 bytes when encoded)
- For login:
    1. Enter a unique **username** or **email,** and **password**
    2. The server will print an **OTP** in the **server terminal:**

       ```
       🔐 OTP for abc: vgpOuX8V   (copy to client)
       ```

    3. Copy that OTP and paste it into the client when prompted

2. On first successful login, your identity keys and prekeys are automatically generated and uploaded.

Note: Sessions expire after 24 hours (you will need to log in again).

```
=== Secure E2EE Instant Messenger ===
1. Login / Register
11. Exit
Choice: 1
New user? (y/n): n
Username or email: 1
Password: 1
Too many failed attempts. Account locked for 30 seconds.

=== Secure E2EE Instant Messenger ===
1. Login / Register
11. Exit
Choice: 1
New user? (y/n): n
Username or email: 1
Password: 1
Account locked. Try again in 27 seconds.
```

Example of too many failed attempts. The UI shows the time required to retry logging in.

4.2 Contact Management

Send friend request (option 2)

- Enter a username to send them a friend request.

```
Choice: 2
Username to request: abc
✅ Friend request sent to abc.
```

(Note: You cannot message a user until they accept your request).

View & respond to friend requests (option 3)

- Choose **3. View & respond to friend requests**
- You will see **Incoming** and **Outgoing** requests
- For incoming: type "y" to **accept** or "n" to **decline**
- You can also cancel your own outgoing requests from this screen

```
--- Incoming Friend Requests ---
From: bca
From: cba

Enter a username to handle your request (or press Enter to skip): bca
Accept (y) or Decline (n) bca's request? y
✅ Accepted! You are now friends.
```

```
Choice: 3

--- Incoming Friend Requests ---
From: cba

Enter a username to handle your request (or press Enter to skip): cba
Accept (y) or Decline (n) cba's request? n
❌ Declined cba's request.
```

```
Choice: 3

--- Incoming Friend Requests ---
No incoming friend requests.

--- Outgoing Friend Requests ---
To: abc

Enter a username to cancel your request (or press Enter to skip): abc
🚫 Cancelled.
```

Remove friend (option 4)

- Enter the name who you want to unfriend

```
Choice: 4
Remove friend username: cba
✅ Removed user cba from friends.
```

Blocking: (option 8)

- Choose 8**. Block user**
- Enter the username to block

```
8. Block user
9. Unblock user
10. Logout
11. Exit
Choice: 8
Block username: abc
✅ User blocked: abc
```

- Blocked users cannot send you messages or friend requests (their requests are ignored)
- Once blocked, the server will immediately halt the delivery of any new messages from them to your client.

```
> test
🔄 First message to abc. Establishing shared secret...
⚠️ WARNING: Identity key for abc has CHANGED!
   Possible key compromise or they reinstalled the app.
🚨 BLOCKING: You previously verified this contact. You must re-verify the new fingerprint!
❌ Send failed: Failed to establish shared secret
>
```

- The blocked user will not be notified that they have been blocked, but their messages will no longer reach you (message status = “sent”).

Unblocking: (option 9)

- Choose 9. Unblock User
- Enter the username to unblock.
- You can receive the recipient's messages and friend requests which are sent after unblocking.

Important: Only accepted friends can exchange messages. Non-friends can only send friend requests.

```
7. Chat with contact
8. Block user
9. Unblock user
10. Logout
11. Exit
Choice: 7
Chat with username: bca
Checking friendship status with bca...
✅ You are friends with bca.
```

```
Chat with username: cba
Checking friendship status with cba...
❌ You are not friends with cba.
   Please send a new friend request if needed.
```

4.3 Viewing Conversations

- Choose **5. List contacts & conversations**
- You will see a list like:
-
```
👤 a  ✅ | Unread: 0 | Last Activity: 2026-04-08T12:28:06
👤 1  | Unread: 0 | Last Activity: 2026-04-07T05:47:00
```
  Conversations are sorted by most recent activity.
- Unread counters are shown per contact.

*Note: Blocking a user does not remove the user from the friend list. Use **4. Remove friend** to do so.*

4.4 Verify Contact Fingerprint

```
Choice: 6
Enter contact username to verify: bca

--- Fingerprint for bca ---
Safety Number: beca884a2047fbce4e3da0e10f531a2b
---------------------------------
Mark as verified? (y/n):
```

```
Choice: 6
Enter contact username to verify: abc

--- Fingerprint for abc ---
Safety Number: beca884a2047fbce4e3da0e10f531a2b
---------------------------------
Mark as verified? (y/n): y
✅ abc has been marked as verified!
```

- **Verify fingerprint for your contact** allows you validate your contact's fingerprint. If both are the same, it can be verified by users that can ensure the safety and security of contacts and messages.
- On the **first message** to a new contact, a secure session is automatically established using X3DH.
- The client will display a **safety number / fingerprint** (32-character hex string).
- **Verify the fingerprint** out-of-band (e.g., in person, via another secure channel) with your contact.
- If the fingerprint matches, type `y` when asked "Mark as verified?"
- If a contact's identity key changes later, you will see a clear **warning**. New messages are blocked until you manually verify the new fingerprint

(recommended policy to prevent potential compromise).

4.5 Chatting

1. From main menu, choose **7. Chat with contact**
2. Enter the **username** of an accepted friend
3. You enter the chat screen:

```
--- Chat with alice (type /quit to exit) ---
>
```

Sending normal messages:

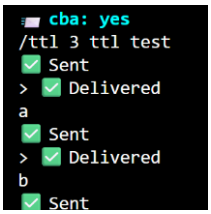
- Just type your message and press Enter

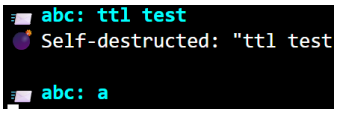
Sending self-destruct (timed) messages

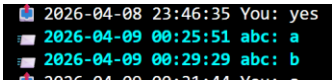
- Use the special command:

  `/ttl <seconds> <message>`

  - Replace `<seconds>` with any number of seconds (e.g., 60, 300, 600)
  - The message is deleted from your UI after the timer expires (client-side), which is set to be 7 days.

- 





Exiting chat:

- Type /quit and press Enter

4.6 Delivery Status

- After sending, you will see:
  - Sent (msg_id=123)
  - Delivered (msg_id=123) — this appears when the recipient's client acknowledges receipt (stronger semantics)
- The server only knows when messages are placed in queues or delivered to online clients (metadata such as timing may be visible to the honest-but-curious server).
- Max size of a message is 10KB.
- The message from blocked user is ignored.

4.7 Offline Messaging

- If your contact is offline, messages are stored as ciphertext on the server.
- When they come online, they are automatically delivered.
- Timed self-destruct TTL is respected on a best-effort basis (server cleans up expired messages periodically).
- Undelivered messages older than 7 days are automatically deleted by the server.

4.8 Logout

- From the main menu, choose **10. Logout**
- Your token is invalidated immediately.

```
Choice: 10
Logged out.
```

4.9 Other Features

- **Unread counters** update automatically.
- **Incremental message loading** is supported on the server (used internally for efficiency).
- **Replay protection** and **duplicate detection** are built into the encryption layer.

5. Security & Privacy Reminders

- **All messages are end-to-end encrypted**. The server never sees plaintext.
- **Self-destruct** only deletes messages from the app and server storage — it cannot prevent screenshots or copying the recipient.
- Always **verify safety numbers** with important contacts.
- If you see an identity key change warning, treat it seriously and re-verify.
- Local state (keys, ratchets, verification flags) is saved in client_state.json in the same folder. Do not share this file.
- If "your local passphrase is unrecoverable" is shown, user should enter what they entered when creating the original .json file.
- Note that the server and network administrators can see who you are messaging (your contact list) and when you are messaging them. (Metadata is visible)

6. Troubleshooting

- **OTP not appearing?** Check the server terminal.
- **"Not friends" error?** Send and accept a friend request first.
- **Decryption failed?** Usually caused by replay (ignored automatically) or session issues — try sending a new message.
- **Server not responding?** Make sure `server.py` is running.
- **Clean start**: Delete `secure_im.db` and `client_state.json` to reset everything.

7. Exiting the Application

- From main menu, choose **11. Exit**
- The client terminates.